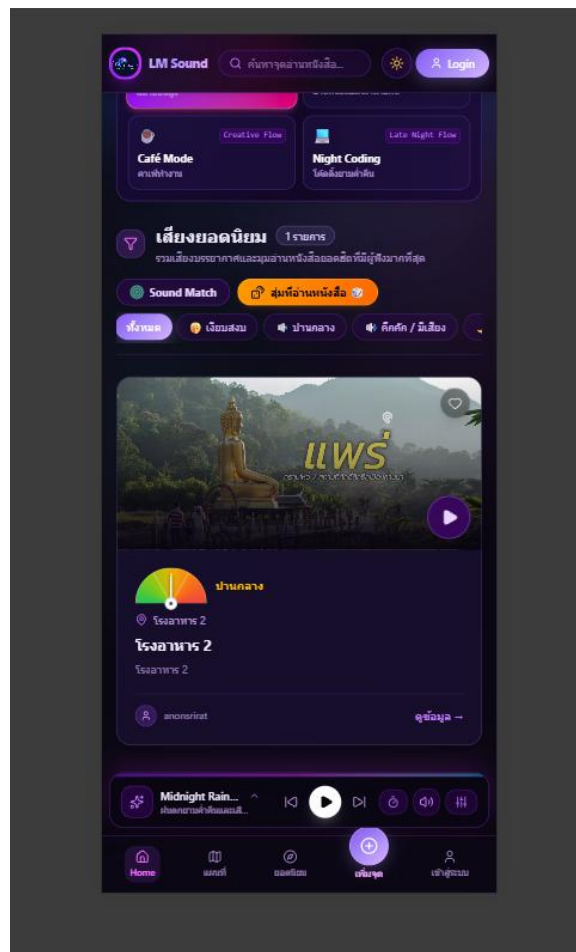


รายงานโครงงานฉบับสมบูรณ์

การออกแบบและพัฒนาเว็บแอปพลิเคชัน LMSound

0214321 Web Application Design and Development Module

ภาคเรียนที่ 1/2569



ระบบแนะนำสถานที่อ่านหนังสือและเสียงบรรยากาศสำหรับการโฟกัส

ชื่อทีม: _____

สมาชิก: กรอกชื่อและรหัสตามที่สมัครจริงก่อนส่ง

วันที่นำเสนอ: Week 15 / 15 กันยายน 2569

บทสรุปผู้บริหาร

LMSound เป็นเว็บแอปพลิเคชันที่ช่วยผู้ใช้ค้นหาและเลือกสถานที่อ่านหนังสือหรือทำงาน

พร้อมฟังเสียงบรรยากาศที่เหมาะสมกับกิจกรรม ระบบรวมแผนที่สถานที่จริง การ์ดข้อมูล สถานะความเงียบ ภาพและเสียงตัวอย่าง สิ่งอำนวยความสะดวก ระบบเสียงผ่อนคลาย การให้คะแนน และการจัดการเนื้อหาสำหรับผู้ดูแลระบบไว้ในแพลตฟอร์มเดียว การออกแบบเน้นการใช้งานบนมือถือและรองรับการติดตั้งแบบ Progressive Web App

โครงการพัฒนาด้วย Next.js App Router, React, TypeScript, Prisma และ PostgreSQL ใช้ JWT ใน HTTP-only cookie สำหรับ session, bcrypt สำหรับรหัสผ่าน และ Zod สำหรับตรวจสอบข้อมูล ระบบ production ทำงานบน Docker บน VPS และใช้ Cloudflare R2 สำหรับไฟล์ที่อัปโหลด โดยฐานข้อมูลเก็บเฉพาะ metadata ของไฟล์

สารบัญโดยสรุป

- 1 ข้อมูลทีมและโครงการ
- 2 ภาพรวมโครงการ
- 3 פיเจอร์หลักที่พัฒนา
- 4 การออกแบบระบบฉบับสมบูรณ์
- 5 Security และ Technical Quality
- 6 Git Workflow และ Deployment
- 7 การทดสอบระบบ
- 8 ภาพหน้าจอระบบจริง
- 9 ปัญหาและบทเรียน
- 10 Reflection รายบุคคล
- 11 ตรวจสอบ Deliverables

ส่วนที่ 1 ข้อมูลทีมและโครงการ

รายการ	รายละเอียด
ชื่อโครงการ	LMSound: Spatial and Ambient Soundscape
ชื่อทีม	_____
GitHub Repository	https://github.com/arnonsrirat/lm_sound
Production URL	https://lmsound.arnoncore.com
วันนำเสนอ	Week 15 / 15 กันยายน 2569

ลำดับ	ชื่อ-นามสกุล	รหัสนิสิต	บทบาทหลัก
1	_____	_____	หัวหน้าทีม / Full-stack และ Deployment
2	_____	_____	สมาชิกทีม / _____
3	_____	_____	สมาชิกทีม / _____

ส่วนที่ 2 ภาพรวมโครงการ

ปัญหาและที่มาของโครงการ

ผู้ใช้ที่ต้องการอ่านหนังสือหรือทำงานมักต้องทดลองเดินทางไปหลายสถานที่โดยไม่มีข้อมูลกลางที่บอกระดับเสียง บรรยากาศ ภาพจริง เสียงตัวอย่าง และสิ่งอำนวยความสะดวก เช่น Wi-Fi แสงสว่าง ปลั๊กไฟแอร์ หรือพัดลม อีกทั้งการเปิดเสียงเพื่อช่วยโฟกัสก็ต้องสลับไปมาระหว่างหลายแอป

ทีมจึงพัฒนา LMSound ให้เป็นพื้นที่กลางสำหรับค้นหาจุดอ่านหนังสือ ดูตำแหน่งจากแผนที่ ฟังเสียงบรรยากาศ และจัดเพลงผ่อนคลาย โดยออกแบบให้ผู้ดูแลระบบจัดการสถานที่ สื่อ รูปภาพ เพลงเพลย์ลิสต์ และอีเมลเว็บไซต์ได้โดยไม่ต้องแก้ไขโค้ด

กลุ่มผู้ใช้เป้าหมายและคุณค่าที่ระบบมอบให้

กลุ่มผู้ใช้	คุณค่าที่ได้รับ
นิสิต นักเรียน และนักอ่าน	ค้นหาจุดอ่านหนังสือที่เหมาะสมกับระดับเสียงและช่วงเวลา พร้อมดูแผนที่และภาพจริง
ผู้ทำงานหรือผู้ต้องการโฟกัส	เปิดเสียงบรรยากาศ ปรับ mixer ตั้งเวลา และเลื่อนตำแหน่งเพลงได้ในหน้าเดียว
ผู้ใช้มือถือ	ใช้งานผ่าน responsive UI และติดตั้งเป็น PWA
ผู้ดูแลระบบ	จัดการสถานที่ สื่อ เพลง เพลย์ลิสต์ สิ่งอำนวยความสะดวก และอีเมล

ส่วนที่ 3 ฟีเจอร์หลักที่พัฒนา

ฟีเจอร์	คำอธิบายสั้นๆ	สถานะ
Spot และแผนที่	สร้าง แก้ไข ลบ ค้นหา พร้อมปักด้าย ภาพ เสียง ระดับเสียง ช่วงเวลา และหมวด	เสร็จ

ฟีเจอร์	คำอธิบายสั้นๆ	สถานะ
แผนที่แบบ Modal	เห็นหมดทั้งหมด หมดที่เลือกไฮไลท์สีฟ้า และการดัดแปลงรายการ	เสร็จ
สิ่งอำนวยความสะดวก	ไอคอน Wi-Fi แสงสว่าง ปลั๊กไฟแอร์ พัดลม และรายการอื่น	เสร็จ
Audio Engine	เล่น หยุด เปลี่ยนเพลง volume seek mixer focus recipes และ sleep timer	เสร็จ
คลังเพลงพ็อนคลาย	เพลง เพลย์ลิสต์ การเผยแพร่ การให้คะแนน และล็อกเนื้อหาสำหรับผู้ไม่ login	เสร็จ
Admin Content Management	จัดการรูป เสียง เพลง playlist และสถานะเผยแพร่	เสร็จ
Authentication	สมัคร ยืนยันอีเมล ลืมรหัสผ่าน รีเซต และ Google Login	เสร็จ
Responsive และ PWA	Bottom navigation โปรไฟล์มือถือ install prompt และ service worker	เสร็จ
Theme และเทศกาล	light/dark โลโก้ favicon และช่วงวันเวลา theme จาก admin	เสร็จ
Upload ที่เสถียร	บีบอัดรูป direct upload R2 progress retry และไฟล์เสียงขนาดใหญ่	เสร็จ

ส่วนที่ 4 การออกแบบระบบฉบับสมบูรณ์

โครงสร้างสถาปัตยกรรมและเทคโนโลยีหลัก

ระบบใช้แนวคิด Controller-Service-Model ในรูปแบบ Next.js App Router: Route Handler ทำหน้าที่เป็น controller รับ request และตรวจสอบสิทธิ์, action/service รวม business logic และ validation, Prisma Client เป็น data access layer และ PostgreSQL เป็นฐานข้อมูลหลัก ส่วนหน้าใช้ React Client Components และ Context สำหรับ audio engine ขณะที่ไฟล์ binary ถูกส่งตรงจาก browser ไป Cloudflare R2 ผ่าน presigned URL

ชั้นระบบ	เทคโนโลยี / หน้าที่
Presentation	Next.js App Router, React 19, TypeScript, Tailwind CSS 4, Leaflet / React Leaflet
Application	Route Handlers, Server Actions, Zod, JWT session, bcryptjs
Data	Prisma Client, PostgreSQL 16, relational models และ migration
Object Storage	Cloudflare R2 ผ่าน AWS SDK S3-compatible API และ presigned PUT URL
Deployment	Docker multi-stage, Docker Compose, Docker Hub และ VPS หลัง

ชั้นระบบ	เทคโนโลยี / หน้าที่
	reverse proxy
External Services	Google OAuth, Resend, OpenStreetMap และ Google Drive integration

โครงสร้างข้อมูลสุดท้าย

โมเดลหลักประกอบด้วย User และ Spot สำหรับตัวตนและสถานที่, MediaAsset สำหรับ metadata ของไฟล์, RelaxationPlaylist และ RelaxationPlaylistTrack สำหรับการจัดเพลง, TrackRating, SpotRating และ SpotFavorite สำหรับการมีส่วนร่วมของผู้ใช้ รวมถึง AuthToken, SiteSetting และ GoogleDriveConnection สำหรับ token ยืนยันตัวตน การตั้งค่าเว็บไซต์ และบริการภายนอก

Model	ความสัมพันธ์และหน้าที่
User	เจ้าของ Spot, ratings, favorites และ auth tokens; มี role USER/ADMIN
Spot	สถานที่อ่านหนังสือ มีพิกัด ภาพ เสียง ระดับเสียง เวลาแนะนำ amenities และ author
MediaAsset	metadata เช่น fileKey, mimeType, size, url, folder, isPublished และสถิติ
RelaxationPlaylist / Track	เพลย์ลิสต์และตารางเชื่อมเพลงพร้อม position
TrackRating / SpotRating / Favorite	คะแนนและรายการถูกใจแบบหนึ่งรายการต่อผู้ใช้ต่อรายการ
SiteSetting	key-value สำหรับโลโก้ แบนเนอร์ สีม ข้อความ และช่วงเวลาเทศกาล
AuthToken / GoogleDriveConnection	token สำหรับ email flow และข้อมูล OAuth ของ Google Drive

API Endpoints ทั้งหมด

Method	Path	ต้อง Login?	คำอธิบาย
GET	/api/auth/me	ไม่บังคับ	ตรวจสอบ session และผู้ใช้ปัจจุบัน
POST	/api/auth/register	ไม่บังคับ	สมัครสมาชิกและยืนยันอีเมล
POST	/api/auth/login	ไม่บังคับ	เข้าสู่ระบบด้วย username/email
POST	/api/auth/logout	ไม่บังคับ	ล้าง session cookie
GET	/api/auth/google และ callback	ไม่บังคับ	Google OAuth
POST	/api/auth/verify-email	ไม่บังคับ	ยืนยันรหัสอีเมล
POST	/api/auth/password-reset	ไม่บังคับ	ขอรหัสและรีเซตรหัสผ่าน
GET/POST	/api/spots	GET ไม่บังคับ / POST login	อ่านและสร้างสถานที่
GET/PATCH/DELETE	/api/spots/[id]	แก้ไขต้อง login	ดู แก้ไข หรือลบตามสิทธิ์
POST	/api/spots/[id]/engagement	ต้อง login	favorite และ rating
GET/POST	/api/relaxation	ให้คะแนนต้อง login	เพลงเพลย์ลิสต์ การฟัง และ rating
GET/POST/DELETE/PATCH	/api/admin/relaxation	ADMIN	จัดการเพลงและ playlist

Method	Path	ต้อง Login?	คำอธิบาย
GET/POST/PATCH/DELETE	/api/admin/media	ADMIN	คลังไฟล์และ metadata
GET/DELETE	/api/admin/media/[fileId]	ADMIN	ดูหรือลบไฟล์
POST	/api/uploads/presign	ต้อง login	ขอ presigned URL
POST	/api/uploads/complete	ต้อง login	ตรวจสอบ object และบันทึก metadata
GET	/api/uploads/status	ต้อง login	ตรวจสอบสถานะอัปโหลด
GET	/api/uploads/[...key]	ตามนโยบายไฟล์	แชร์ไฟล์ที่อนุญาต
GET	/api/admin/google-drive/*	ADMIN	เชื่อมต่อและตรวจสอบ Google Drive

จุด Authorization ที่ทำจริง

- ผู้ใช้ทั่วไปดูสถานที่และเพลงที่เผยแพร่ได้ แต่ต้อง login ก่อนฟังเพลงออนไลน์ ให้คะแนน กดถูกใจ หรือสร้าง/แก้ไขข้อมูล
- เจ้าของสถานที่เท่านั้นที่แก้ไขหรือลบ Spot ของตนเองได้ ส่วน ADMIN จัดการ Spot และสื่อได้ทั้งหมด
- Route ของ admin ใช้ requireAdmin และ action สำคัญใช้ requireAuth ก่อนแตะฐานข้อมูล
- Server เป็นผู้สร้าง fileKey และตรวจสอบ userId, MIME type, size และ object ใน R2 ไม่เชื่อ path หรือ owner จาก client

ส่วนที่ 5 Security และ Technical Quality

	มาตรการ	สถานะ
☑	รหัสผ่านเข้ารหัสด้วย bcrypt ไม่เก็บ plain text	ทำจริง
☑	ใช้ Prisma Client ไม่มี raw SQL ที่เสี่ยง SQL Injection	ทำจริง
☑	ไม่ใช้ dangerouslySetInnerHTML กับข้อมูลผู้ใช้โดยไม่ sanitize	ทำจริง
☑	ตรวจสอบข้อมูลนำเข้าด้วย Zod ก่อนบันทึก	ทำจริง
☑	มี Authorization ป้องกันแก้ไข/ลบข้อมูลของคนอื่น	ทำจริง
☑	R2/OAuth/Resend secrets อยู่ฝั่ง server	ทำจริง
☑	session ใช้ HTTP-only cookie และ JWT อายุจำกัด	ทำจริง
☑	อัปโหลดใช้ presigned URL และตรวจสอบ MIME/size	ทำจริง

ส่วนที่ 6 Git Workflow และ Deployment

ทีมใช้ main เป็น branch production และแยกงานใน anon กับ Ninja สมาชิก push งานของตนเองขึ้น origin แล้วหัวหน้าทีม merge เข้า main ก่อน deploy ขั้นตอน production คือ build Docker multi-stage, tag image ตาม commit และ latest, push Docker Hub แล้วให้ VPS pull image ใหม่และ recreate เฉพาะ app container โดยคง PostgreSQL และ volume uploads ไว้

ขั้นตอน	สิ่งที่ทำจริง
Development	พัฒนางานบน branch anon หรือ Ninja
Integration	merge งานเข้า main และ push origin/main
Build	สร้าง image arnonsrirat/lmsound ด้วย Dockerfile multi-stage
Registry	push tag commit และ latest ไป Docker Hub
VPS	docker compose pull และ up -d --no-deps lmsound
Verification	ตรวจ logs และ curl production ให้ได้ HTTP 200

ปัญหาระหว่าง deploy ได้แก่ database connection หลุดและพื้นที่ดิสก์ VPS ลดลงจาก Docker image/logs จึงตรวจ connection, cleanup image/logs และดูแล database container อย่างระมัดระวัง ส่วนการอัปเดตมือถือแก้ด้วย compression, queue/status, retry และ direct upload ไป R2

ส่วนที่ 7 การทดสอบระบบ

กรณีทดสอบ	วิธีตรวจ	ผล
สมัคร/ยืนยันอีเมล	สมัคร รับรหัส และยืนยัน	ผ่าน
Login/Logout	เข้าสู่ระบบและตรวจ session	ผ่าน
Google Login	ทดสอบ OAuth callback และ production redirect	ผ่านเมื่อ env ถูกต้อง
ลิ้งรหัสผ่าน	ขอรหัส Resend และตั้งใหม่	ผ่านเมื่อ env พร้อม
Spot CRUD	สร้าง แก้ไข ลบ และ owner guard	ผ่าน
แผนที่	มือถือ/desktop เห็นหมุดและเลือกหมุด	ผ่าน
Audio	เล่น หยุด เปลี่ยน volume/mixer/seek	ผ่าน
Upload	รูปหลายไฟล์และเสียงใหญ่พร้อม progress	ผ่านหลังเพิ่ม R2 flow
Admin	media เพลง playlist theme Drive	ผ่าน
Build/Deploy	tsc build Docker push pull HTTP health	ผ่าน

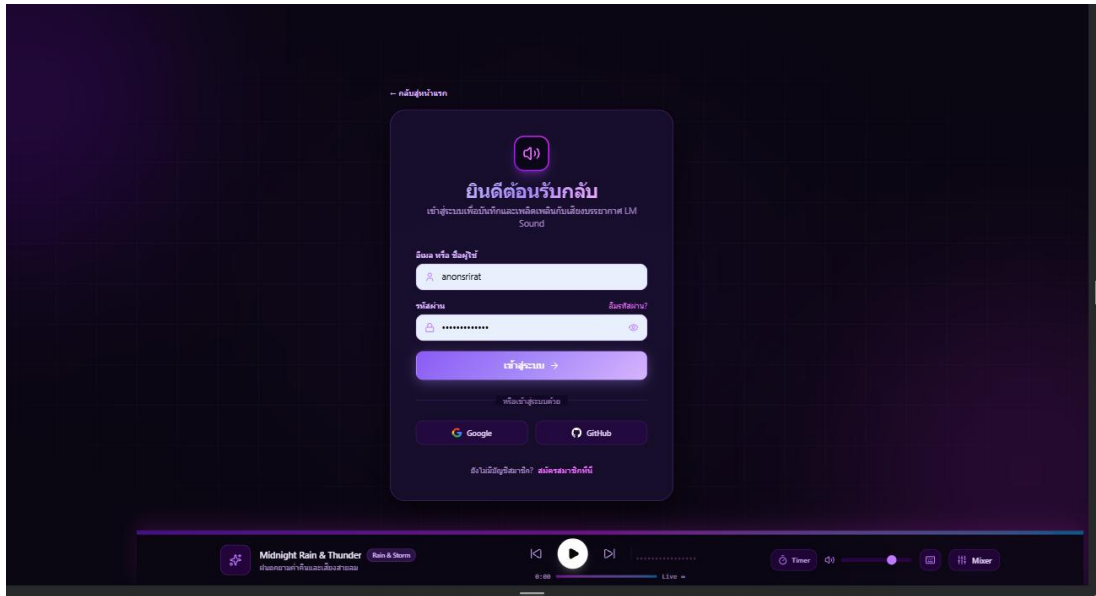
ส่วนที่ 8 ภาพหน้าจอหน้าหลักของระบบจริง

หน้าจอที่ 1 หน้า Home และรายการสถานที่



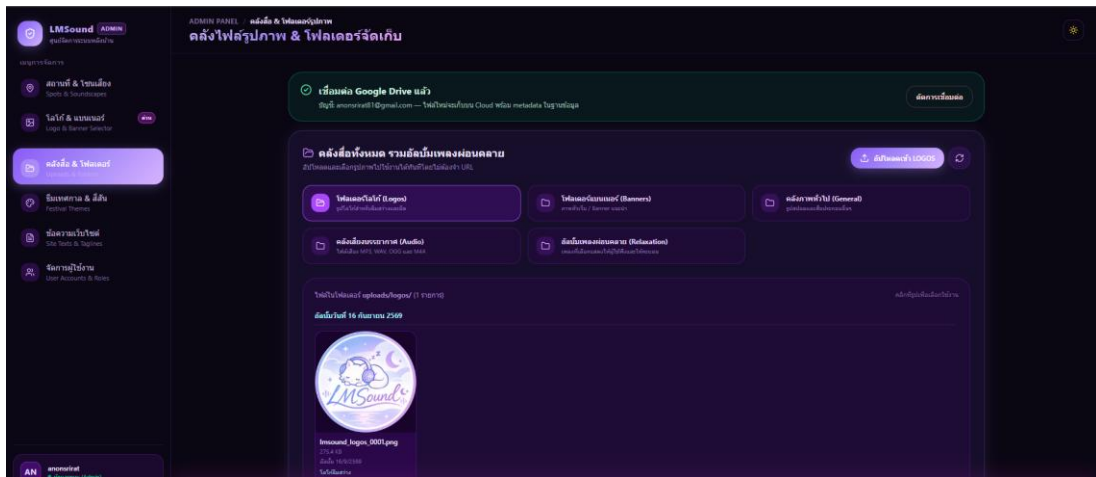
หน้าแรกแสดง banner, soundscape, filter, การ์ดสถานที่ และ audio player

หน้าจอที่ 2 หน้า Login และ Authorization



หน้า login พร้อม password และ provider ภายนอก

หน้าจอที่ 3 Admin Media Library



หน้า admin สำหรับจัดการคลังรูปภาพ ไฟล์เดอรั และสื่อ

ส่วนที่ 9 ปัญหาที่พบและบทเรียนที่ได้

ปัญหาและวิธีแก้

- ฐานข้อมูลหรือ Prisma เคยมีปัญหานจนข้อมูลสถานที่ไม่แสดง ทีมจึงตรวจ connection, migration, container และแยก fallback ออกจาก production data

- การโหลดรูปและอัปโหลดหลายไฟล์ทำให้มือถือค้าง จึงใช้ lazy loading, compression, queue/status, retry และ direct upload ไป R2
- แผนที่ใน modal เคยเลื่อนหรือแสดงผลผิด จึงเพิ่ม invalidateSize, bounds, marker ทุกจุด, double-click สำหรับปิกหมด และจัดการคาร์ทใหม่
- เสียงและ modal เคยทับกันบนมือถือ จึงใช้ portal, z-index ชัดเจน, ซ่อน player เมื่อเปิด editor/profile และใช้ toast แทน window.alert
- ธีม light/dark เคยอ่านยาก จึงกำหนด palette, logo light/dark, favicon sync และตั้งค่าจาก admin

สิ่งที่เรียนรู้และสิ่งที่จะพัฒนาต่อ

ทีมได้เรียนรู้การออกแบบ full-stack ตั้งแต่ requirement, responsive UI, relational data modeling, authorization, OAuth, email workflow, object storage, Docker และ deployment จริง การแก้ปัญหาจากมือถือทำให้เห็นความสำคัญของ performance budget, network behavior และ progressive enhancement

หากมีเวลาเพิ่ม ควรเพิ่ม automated end-to-end tests, monitoring/error tracing, multipart resumable upload, image CDN transformation, offline cache และระบบแก้ไข profile/email แบบยืนยันอีเมลครบวงจร

ส่วนที่ 10 Reflection รายบุคคล

สมาชิกคนที่ 1

หัวข้อ	คำตอบของสมาชิก
ชื่อ-นามสกุล / รหัสนิสิต	_____
บทบาท/สิ่งที่รับผิดชอบหลัก	_____
สิ่งที่ได้เรียนรู้ / ทักษะที่พัฒนาขึ้น	_____ _____ _____
ความท้าทายส่วนตัวและวิธีการจัดการ	_____ _____ _____

สมาชิกคนที่ 2

หัวข้อ	คำตอบของสมาชิก
ชื่อ-นามสกุล / รหัสนิสิต	_____
บทบาท/สิ่งที่รับผิดชอบหลัก	_____
สิ่งที่ได้เรียนรู้ / ทักษะที่พัฒนาขึ้น	_____

หัวข้อ	คำตอบของสมาชิก

ความท้าทายส่วนตัวและวิธีจัดการ	_____ _____ _____

สมาชิกคนที่ 3

หัวข้อ	คำตอบของสมาชิก
ชื่อ-นามสกุล / รหัสนิสิต	_____
บทบาท/สิ่งที่รับผิดชอบหลัก	_____
สิ่งที่ได้เรียนรู้ / ทักษะที่พัฒนาขึ้น	_____ _____
ความท้าทายส่วนตัวและวิธีจัดการ	_____ _____

ส่วนที่ 11 ตรวจสอบ Deliverables ก่อนส่ง

- ☐ ลิงก์ Production Deployment: <https://lmsound.arnoncore.com>
- ☐ GitHub Repository: https://github.com/arnonsrirat/lm_sound
- ☐ README.md อธิบายโปรเจกต์และวิธีติดตั้ง
- ☐ สไลด์นำเสนอสำหรับ Demo Day
- ☐ รายงานฉบับสมบูรณ์และ Reflection ของสมาชิกทุกคน

สมาชิกคนที่ 1	สมาชิกคนที่ 2	สมาชิกคนที่ 3
ลงชื่อ _____	ลงชื่อ _____	ลงชื่อ _____
วันที่ _____	วันที่ _____	วันที่ _____